

20250709 日报

今日工作内容

1. 七彩石故障兜底的研究：yaml 文件进行兜底
 - 1、今日研究进度：
 - a) 新增的 standbyrainbow 包源码如下：

```

var ServerConfigPath = defaultConfigPath
var gCfg atomic.Value

const (
    defaultConfigPath =
        "./standbyrainbow_conf/standbyrainbow.yaml"
)

func Init() {
    yamldata, err := stanbyRainbowLoadConfig(ServerConfigPath)
    if err != nil {
        panic("load config fail: " + err.Error())
    }
    err = setStandbyRainbow(yamldata)
    if err != nil {
        panic("set config fail: " + err.Error())
    }
}

func stanbyRainbowLoadConfig(configPath string) ([]byte, error) {
    data, err := os.ReadFile(configPath)
    if err != nil {
        return nil, err
    }
    return data, nil
}

func setStandbyRainbow(yamldata []byte) error {
    cfg := rainbow.GetDefaultConfig()
    if err := yaml.Unmarshal(yamldata, cfg); err != nil {
        return err
    }
    gCfg.Store(cfg)
    return nil
}

// GetConfig 获取配置
func GetConfig() *rainbow.Config {
    return gCfg.Load().(*rainbow.Config)
}

```

b) 使用方法:

- i) 在 gdp 平台添加七彩石配置信息, 默认挂载路径为"./standbyrainbow_conf/standbyrainbow.yaml";
- ii) 七彩石故障后, 可直接在 yaml 中修改配置信息;
- iii) 导入 standbyrainbow 包, 并在外部需要拉取七彩石配置的地方 GetConfig()即可, 如:

```
import (  
    pb "git.woa.com/drewjjli/test/proto"  
    "git.woa.com/drewjjli/test/standbyrainbow"  
)  
  
func main() {  
    s := trpc.NewServer()  
    standbyrainbow.Init()  
    cfg := standbyrainbow.GetConfig()  
    pb.RegisterGreeterService(s.Service("hello.Greeter"),  
    &greeterImpl{})  
    if err := s.Serve(); err != nil {  
        log.Fatal(err)  
    }  
}
```

c) 自测结果:

```
test-dev-v1-784495df77-g6r15-----[容器test-dev-v1日志]-----  
2025/07/09 11:54:37 ip.go:125: [0:36m[debug] set local ip from server: 30.163.240.131:8080  
2025/07/09 11:54:37 ip.go:144: [0:37m[info] local ip: 21.13.9.62  
[0:40m[info] [PersonInfo{Name:drewjjli Age:24} Hobbies:[Basketball Musicrap]] tnet/server.transport.go:100 service: hello.Greeter is using tnet tcp transport, current number of pollers: 1  
2025-07-09 11:54:38.076 INFO server/service.go:207 process: 1, trpc service: hello.Greeter launch success, tcp: 21.13.9.62:8080, serving ...  
2025-07-09T11:54:38+08:00 [34mINFO[0m tnet@v0.1.0/tcpserver.go:88 tnet tcp service started, current number of pollers: 1, use tnet.SetNumPollers to change it
```

结论: 此方法研究偏了, 与真正的需求不太吻合, 后续需要修改到完全解耦: 只需要在 yaml 中控制开关+代码中导入包即可。

2. 进一步研究, 新建插件的方法:

```

    // Package standbyrainbow 从备用七彩石获取配置
package standbyrainbow

import (
    "errors"
    "fmt"
    "os"
    "time"

    "git.code.oa.com/rainbow/golang-sdk/log"
    "git.code.oa.com/trpc-go/trpc-go/plugin"
)

const PluginType = "standbyrainbow"

func init() {
    fmt.Println(111)
    plugin.Register(PluginType, &Plugin{})
    log.Info("Plugin standbyrainbow registered") // 确认注册日志
}

// Config 插件配置
type Config struct {
    Enable bool        `yaml:"enable"`
    Name   string          `yaml:"name"`
}

// Plugin 插件
type Plugin struct {
}

// Type 插件的类型
func (p *Plugin) Type() string {
    return PluginType
}

// Setup 装载插件
func (p *Plugin) Setup(name string, dec plugin.Decoder) error {
    fmt.Println(666)
    if dec == nil {
        return errors.New("standbyrainbow config decoder empty")
    }
}

```

进一步思考：这是新建插件的方法，感觉可以直接在 config-rainbow 插件下新起一个字段，然后覆盖 trpc-go 中 config-rainbow 包中的注册函数和 Setup()函数即可，但还需后续验证。

明日待办：

1. 七彩石故障的兜底研究；